DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

MySQL gives away its thread pool, and Aurora DSQL learns foreign keys without locks

Issue #11 | Thu, 15 Oct 2026 | ⏱️ ~40 min read | Level 3/5

🗄️ Today's two stories

1. **MySQL's thread pool, long an Enterprise-only feature, now ships in the free Community edition, and the first independent benchmarks are in.** Oracle quietly moved it in July with MySQL 26.7.0. On 5 and 12 October, Percona published a two-part benchmark: with thousands of connections on an I/O-bound server, a well-tuned pool delivered up to 17.9 times the throughput, but with few connections it slowed things down. We explain why one-thread-per-connection breaks, how the pool schedules work, how MySQL's pool differs from Percona's and MariaDB's, and how to decide whether to turn it on.
2. **Amazon Aurora DSQL, AWS's distributed Postgres-compatible database, spent its summer closing the gap with real Postgres: foreign keys, partial indexes, batched joins and more.** Foreign keys are the interesting one, because DSQL has no locks. AWS explained on 1 October how it enforces them anyway, and announced partial indexes on 2 October. We explain how DSQL commits work, why a foreign key check is hard in a lock-free database, the trick AWS used, and what will now fail with a retryable error in your app.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 MySQL's thread pool goes free

MYSQL 26.7.0 · COMMUNITY EDITION · RELEASED 28 JUL 2026 · PERCONA BENCHMARKS 5 AND 12 OCT · ~18 MIN READ

Imagine a restaurant that hires a new waiter for every customer who walks in. At ten customers it's luxurious. At two thousand, the waiters spend all their time bumping into each other in the kitchen doorway, and nobody gets fed. That's how MySQL has always handled connections by default. For years the fix, a "thread pool" that lets a small crew of waiters serve everyone, was something you had to pay Oracle for. This summer Oracle started giving it away, and this week the first serious benchmarks of the free version landed. The results are spectacular in one corner and disappointing in another, and the reasons why tell you a lot about how a database server really spends its time.

WHAT HAPPENED

On **28 July 2026**, Oracle shipped three MySQL server releases at once: **MySQL 26.7.0**, the first release to use a new calendar-style version number, plus bug-fix releases **9.7.2 LTS** and **8.4.11 LTS**.[1] Tucked into the 26.7.0 release notes, under "Thread Pool Notes", is one line: "MySQL Thread Pool plugin, formerly only available in MySQL Enterprise Edition, is now available in MySQL Community Edition 26.7.0. (WL #17296)".[2] Oracle's announcement post repeats it: "The MySQL Thread Pool Plugin is now available in MySQL Community Edition."[1]

Both 26.7.0 and 9.7.2 also changed one default: `thread_pool_max_unused_threads` went from 2 to 32.[2,3]

The feature got little attention until Percona engineer **Bogdan Degtyariov** published a two-part benchmark. Part 1, on **5 October**, measured Percona Server's own thread pool and found it up to **17.9 times** faster than no pool on an I/O-bound workload with 2,560 connections.[4] Part 2, on **12 October**, put MySQL's newly free pool head-to-head with Percona's and found that neither wins everywhere.[5] The same day, Percona's Anil Joshi published an explainer of the 26.7 release that called the thread pool "the good news" of the release.[6]

A quick word on the version numbers, because they're new. MySQL now uses `Year.Month.Patch`, so 26.7.0 means July 2026. It's an **Innovation release** ≈ *a MySQL release line that gets new features quickly but is supported only until the next one, as opposed to a long-term support (LTS) line*, and the last LTS was 9.7, released in April 2026. Percona says the next Innovation release is tentatively 26.10.0 and the next LTS is expected in April 2028 as 28.4.0, and that you can only reach 26.7 by going through 9.7 first.[6]

THE BACKGROUND YOU NEED

One thread per connection

When a client connects to MySQL, the server by default creates a dedicated operating-system **thread** ≈ *an independent stream of execution inside a process, with its own stack, scheduled onto CPU cores by the operating system* for it. That thread reads the client's statements, runs them and sends back results, and it lives as long as the connection does. MySQL's documentation describes the model and its limits plainly: "The default thread-handling model in MySQL Server executes statements using one thread per client connection. As more clients connect to the server and execute statements, overall performance degrades."[7]

The docs list three reasons why:[7]

1. "Too many thread stacks make CPU caches almost useless in highly parallel execution workloads." Every thread has its own stack memory, and when thousands of them take turns on the CPU, each one evicts the others' data from the **CPU cache** ≈ *small, very fast memory on the processor chip that holds recently used data; a miss means a much slower trip to main memory*.
2. "With too many threads executing in parallel, **context switching** ≈ *the operating system pausing one thread, saving its state and loading another's so it can run on the same core; each switch costs CPU time and pollutes caches* overhead is high."
3. "Too many transactions executing in parallel increases resource contention. In InnoDB, this increases the time spent holding central **mutexes** ≈ *locks inside the server's own code that let only one thread at a time touch a shared*

data structure."

The third point is the subtle one. Running 2,000 queries at once is not faster than running 80 at a time on an 80-core machine. The extra ones just queue up inside the database on internal locks, where waiting is expensive and unfair, instead of queueing outside in an orderly line.

How do you know you're there? MySQL's FAQ gives a rule of thumb: watch the `Threads_running` status variable, which counts statements executing right now. If it "consistently exceeds a region where the server won't operate optimally (usually going beyond 40 for InnoDB workloads), the Thread Pool should be beneficial."[8]

Your app's connection pool is a different thing

If you use HikariCP, a Go `sql.DB` or Django's persistent connections, you already have a "connection pool". That's on the client side, and it solves a different problem. MySQL's FAQ draws the line: a client-side pool caches idle connections "to ensure that a MySQL client does not constantly connect to and disconnect from the MySQL server", but it "has no visibility as to the query handling capabilities or load of the back-end MySQL server." A **thread pool** *≈ a server-side mechanism that runs statements from many client connections on a small, managed set of worker threads, instead of one thread per connection* works inside the server and decides how many statements run at once. The two are "orthogonal and can be used independent of each other."[8]

The trouble is that client pools multiply. Fifty application instances, each with a pool of 40, is 2,000 connections, and each of them is a server thread.

Who had a thread pool before

MySQL's thread pool has been sold as part of **MySQL Enterprise Edition** *≈ Oracle's paid, commercially supported edition of MySQL, which adds plugins such as the thread pool, audit logging and backup tools*. The 9.7 manual still files it under that heading: "MySQL Enterprise Edition includes MySQL Enterprise Thread Pool, implemented using a server plugin."[9]

If you wanted pooling for free, you had to switch servers. MariaDB has shipped one since its 5.1–5.3 releases, and "MariaDB 5.5 introduced an improved thread pool." MariaDB's is built into the server rather than being a plugin, and uses the operating system's best mechanism for watching many network connections (`epoll` on Linux, the native thread pool on Windows).[10] Percona Server for MySQL, a free drop-in replacement for MySQL, also has one built in.[11] Percona's part 1 also mentions third-party plugins.[4]

Oracle has been moving Enterprise features into Community for a while. When MySQL 9.7 LTS shipped in April, an Oracle blog post titled "MySQL 9.7 Is Out and the Community Wins" listed replication metrics components, a telemetry component, writable JSON duality views and the hypergraph optimizer as newly free.[12] The thread pool is the latest, and the one with the most direct effect on performance.

HOW IT WORKS

Thread groups, a listener and two queues

The thread pool breaks the link between a connection and a thread. "The thread pool separates connections and threads, so there is no fixed relationship between connections and the threads that execute statements received from those connections."[13]

Here's what happens when your connection sends a query:

1. **Your connection belongs to a group.** The pool is split into **thread groups** *≈ independent sub-pools inside the thread pool, each managing a share of the connections; MySQL's default is 16*. "As connections are established, the thread pool assigns them to thread groups in round-robin fashion." The default number of groups is 16.[13]
2. **A listener notices.** "Each thread group has a **listener thread** *≈ a thread in each group that waits for any of the group's connections to send a statement, instead of one thread per connection waiting* that listens for incoming statements from the connections assigned to the group."[13]
3. **Run now, or queue.** If nothing else is running or queued in the group, the statement runs immediately. Otherwise it's queued.[13]
4. **Transactions in progress go first.** There's a high-priority queue and a low-priority queue. "The first statement in a transaction goes to the low-priority queue. Any following statements for the transaction go to the high-priority queue if the transaction is ongoing."[13] The idea is to finish transactions that already hold locks before starting new ones.
5. **Nobody starves forever.** A statement that waits too long in the low-priority queue is promoted, controlled by `thread_pool_prio_kickup_timer`, at most "one statement per 10ms (100 per second)" per group.[13]
6. **A worker runs it, then goes back for more.** Workers don't exit after a statement. They pick up the next queued one.

One thread per connection vs a thread pool



Illustrative. Group default (16), queue rules and the per-group query-thread setting come from MySQL's Thread Pool Operation docs.

The hard part: when a query blocks

A pool that only lets one or two statements run per group has an obvious problem. What if the running statement is waiting on a disk read or a row lock? The whole group would sit idle.

MySQL handles this two ways. First, the server code that waits on disk I/O and on **row locks** *≈ locks InnoDB takes on individual rows so two transactions can't change the same row at the same time* calls back into the pool: "there are callbacks to the thread pool to ensure that the thread pool can immediately start a new thread in this group to execute another statement."[13]

Second, for waits the callbacks don't cover, there's a timer. A statement that runs longer than `thread_pool_stall_limit` counts as **stalled** *≈ in MySQL's thread pool, a statement that has run longer than the stall limit, after which it no longer stops others in its group from starting*. "If the statement executes past the stall time, it is permitted to continue but no longer prevents other statements from starting." The default is 60 ms, the setting is in 10 ms units, and the maximum is 600, or six seconds.[13,14] A background thread "regularly monitors thread group states" so no group gets stuck.[13]

That's why the pool is a bet. It works best when most statements are short. The docs recommend it "if your workload comprises mainly short queries", and MariaDB's docs warn it's a poor fit for "many concurrent, long, non-yielding queries" like data-warehouse work.[8,10]

Capping transactions, not just statements

In MySQL 8.0, Oracle added a second throttle, `thread_pool_max_transactions_limit`. An Oracle engineer's 2023 post says it "revitalizes the thread pool", because the original design "addressed the needs of MySQL 5.6" and later InnoDB improvements had eaten into its benefit. With the limit set, each group runs at most `limit / thread_pool_size` transactions at once, rounded up, and a worker sticks with a transaction's statements until it commits or rolls back.[15]

There's an emergency exit. If every slot is held by a long transaction, the server can look frozen, so a connection with the **TP_CONNECTION_ADMIN** *≈ a MySQL privilege that lets a connection bypass the thread pool's transaction limit, run in a separate Admin thread group, and fix things* privilege "ignores the limit" and runs in an "Admin thread group", where it can raise the limit or kill blockers. Nobody has it by default.[13]

Turning it on

MySQL's pool is a plugin you load at startup:[16,6]

```
[mysqld]
plugin-load-add=thread_pool.so
```

Then check that it's active with `SELECT PLUGIN_NAME, PLUGIN_STATUS FROM INFORMATION_SCHEMA.PLUGINS WHERE PLUGIN_NAME LIKE 'thread%';`. When it loads, `thread_handling` changes to `loaded-dynamically`, and three Performance Schema tables appear: `tp_thread_group_state`, `tp_thread_group_stats` and `tp_thread_state`.[16]

The tuning guide's starting point for InnoDB is: `thread_pool_size` equal to the number of physical cores (up to 512, settable only at startup), `thread_pool_algorithm=1` "for high concurrency", `thread_pool_max_transactions_limit` at physical cores × 32, then lowered while you watch throughput, and `thread_pool_query_threads_per_group=2`.[14] To check whether your stall limit fits your queries, it suggests:[14]

```
SELECT SUM(STALLED_QUERIES_EXECUTED) / SUM(QUERIES_EXECUTED)
FROM performance_schema.tp_thread_group_stats;
```

Percona Server and MariaDB do it differently. Their pool is built in and switched on with a server variable, not a plugin, and the knobs have different names and defaults.

Enabling a thread pool: MySQL vs Percona Server

Percona Server (built in)

```
[mysqld]
thread_handling=pool-of-threads
# groups: default = no. of CPUs
thread_pool_size=10
# active threads per group (3)
thread_pool_oversubscribe=4
# stall: milliseconds (500)
thread_pool_stall_limit=500
```

MySQL 26.7 Community (plugin)

```
[mysqld]
plugin-load-add=thread_pool.so
# groups: default 16
thread_pool_size=40
thread_pool_algorithm=1
thread_pool_query_threads_per_group=2
# stall: 10 ms units (default 6)
thread_pool_stall_limit=6
```

Defaults from the Percona Server and MySQL docs. Values shown are examples, not recommendations: the left pane uses Percona's best I/O-bound setting from part 1, the right uses MySQL's tuning guide for a 40-core host.

Percona's docs are explicit that its implementation "is not compatible with upstream": it's "built into the server" where "upstream implements the thread pool as a plugin", and it "does not minimize the number of concurrent transactions".[11] Percona's default for `thread_pool_stall_limit` is 500 ms, and `thread_pool_oversubscribe` (default 3) sets how many workers can be active per group.[11]

WHAT THE BENCHMARKS FOUND

The setup

Percona's tests used a two-socket Intel Xeon Gold 6230 machine (2 × 20 cores, 80 logical CPUs) with 187 GiB of RAM and an NVMe SSD, on Ubuntu 24.04. The workload was **sysbench** *≈ a widely used open-source benchmarking tool; its OLTP read-write test runs short transactions of point selects, range scans, updates, deletes and inserts* OLTP read-write against a 24 GB data set (100 million rows across 20 tables), at 40 to 5,120 concurrent connections. Each run had a 10-minute warm-up and a 15-minute measurement, and **each configuration ran once**.[4]

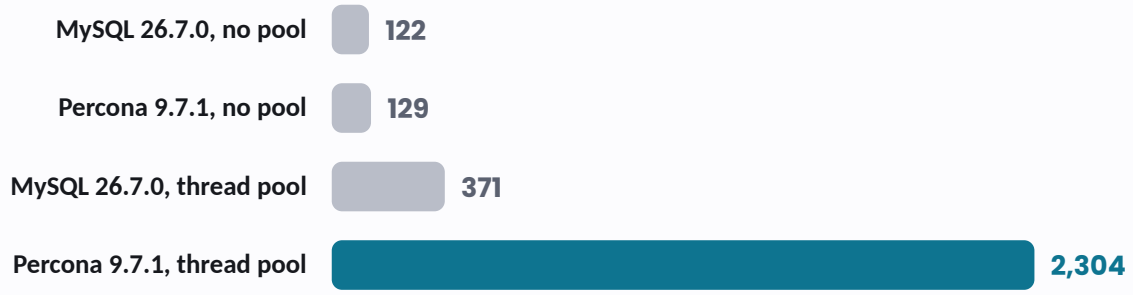
The key variable was the **InnoDB buffer pool** *≈ InnoDB's in-memory cache of table and index pages; if your working data fits, reads never touch disk*. At 2 GB (a 1:12 ratio to the data) the server was I/O-bound. At 12 GB it was partly cached, and at 32 GB everything fit in memory.[4]

When it shines: lots of connections, lots of disk waits

With a 2 GB buffer pool and 2,560 connections, Percona Server with no pool managed 129 transactions per second. With its best pool settings (10 groups, oversubscribe 4) it did 2,304: "17.9 times faster". It peaked at 2,485 TPS at 640 connections and lost only 7.5% going to 2,560. The no-pool server's throughput "drops sharply" from about 120 connections.[4]

MySQL's newly free pool helped too, but less. Same 2 GB setup, 2,560 connections: 371 TPS with the pool and 122 without, about three times better. Its best `thread_pool_size` was 10, smaller than the 40 physical cores.[5]

Sysbench OLTP R/W, 2,560 connections, 2 GB buffer pool (TPS)



Best-tuned configurations from Percona's part 1 and part 2. Single runs on one machine; Percona makes Percona Server.

When it doesn't: few connections, or everything in memory

Part 1's headline conclusion is the caveat: "Thread pooling does not bring any gains and even causes a small slow down on a low number of server connections." With all data in memory (32 GB), Percona Server without its pool won "at every connection count", and every configuration converged at around 16,000 TPS above 640 connections.[4]

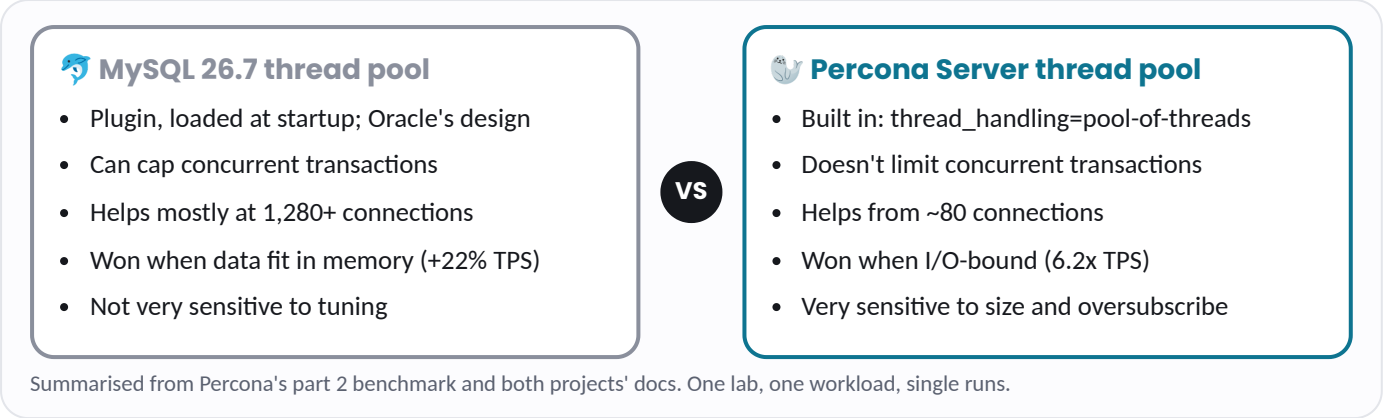
Even there, the pool helped the slowest users. At 2,560 connections, the slowest 5% of transactions without a pool took longer than 787 ms (**p95 latency** ≈ *the time within which 95% of requests finish; the other 5% are slower*), while pooled runs stayed between 235 and 297 ms. The author's point: even when the pool doesn't raise throughput, it "allows to minimize the waiting time for slowest transactions."[4]

MySQL's pool vs Percona's

Part 2 found the two implementations behave quite differently, even though they're built on the same server code:[5]

- **I/O-bound (2 GB):** Percona's pool did 2,304 TPS at 2,560 connections against MySQL's 371, a factor of 6.2. MySQL was faster at 40 connections.
- **Fully cached (32 GB):** the result flipped. MySQL's pool did 19,054 TPS against Percona's 15,519, 22% ahead, and its p95 was about 177 ms against Percona's 235 ms. Unlike Percona's, MySQL's pool still beat no pool here: 18,975 against 17,011 TPS, with p95 dropping from 759 to 153 ms.

- **Tuning:** Percona's pool is "very sensitive" to its two main settings; MySQL's mostly isn't. The best Percona setting at 32 GB was 160 groups, while MySQL's was still 10.
- **Where it starts to help:** MySQL's pool mainly helps at 1,280 and 2,560 connections. Percona's helps from about 80.



Summarised from Percona's part 2 benchmark and both projects' docs. One lab, one workload, single runs.

WHAT IT MEANS FOR YOU

If you run MySQL Community and have connection storms, this is worth testing now. The classic signs are hundreds or thousands of mostly idle connections, `Threads_running` regularly above about 40, and throughput that falls off a cliff when traffic spikes instead of levelling off.[8] Fleets of small app instances, each with its own client pool, are the typical cause.

If your data fits in memory and you have modest connection counts, expect little. Both of Percona's articles say the same thing in different ways: the pool "is not a universal database booster".[4]

Test with your own workload, at your own connection counts. The gap between the best and worst settings was large. In part 1, simply changing `thread_pool_oversubscribe` from 4 to 2 cost 11%.[4] Start from the manual's recommendations, then measure.[14]

Watch the tail, not just the average. Even where the pool didn't increase TPS, p95 latency at high concurrency fell by a factor of three or more.[4,5] If your problem is a few users timing out during spikes, that's the number that matters.

If you're choosing between MySQL and Percona Server or MariaDB, the thread pool used to be a reason to switch. It's now a smaller one, though Percona's results suggest its pool is still stronger when you're I/O-bound.

💥 Watch out for

- **Check that your build has it.** The 26.7.0 release notes say Community 26.7.0. Percona says Community 9.7.2 has it too, but the 9.7.2 release notes don't mention it, and the 9.7 manual still says "For MySQL 9.7, the Thread Pool plugin is included in MySQL Enterprise Edition".[4,3,8] Look for `thread_pool.so` in your plugin directory before you plan around it.
- **It needs a restart.** The plugin loads at startup, and `thread_pool_size` can't change at runtime.[16,14] Switching `thread_pool_max_transactions_limit` from zero to non-zero also needs a restart.[15]
- **A transaction cap can look like an outage.** With the limit set, new work waits when all slots are busy: "No new threads are created to handle new requests."[15] Grant `TP_CONNECTION_ADMIN` to an operator account before you need it.[13]
- **Long queries and pools don't mix well.** Reports and batch jobs that run for seconds will be treated as stalled. If you have many of them, raise `thread_pool_stall_limit` or keep them on a separate server.[14,10]
- **The benchmark numbers are one lab's.** Single runs, one workload, one machine, published by the company that makes Percona Server. Part 2 also gives two different p95 figures for MySQL's pool at 32 GB (153 ms and 177 ms) without explaining the difference.[5]

WHAT HAPPENS NEXT

Percona says the next Innovation release, tentatively 26.10.0, is due later this year.[6] Its release notes will show whether Oracle keeps moving Enterprise features into Community. Also watch the manual: the 26.7 section is titled "MySQL Thread Pool", while the 9.7 one is still "MySQL Enterprise Thread Pool".[7,9] And if your company uses a managed MySQL service, check whether it exposes the plugin. The release notes say nothing about cloud providers.

📖 Words from this story

Innovation release: a MySQL release line that gets new features quickly but is supported only until the next one, unlike an LTS line.

Thread: an independent stream of execution inside a process, with its own stack, scheduled onto CPU cores by the operating system.

CPU cache: small, very fast memory on the processor that holds recently used data.

Context switching: the OS pausing one thread and loading another onto the same core; each switch costs CPU time and pollutes caches.

Mutexes: locks inside the server's own code that let only one thread at a time touch a shared data structure.

Thread pool: a server-side mechanism that runs statements from many connections on a small, managed set of worker threads.

MySQL Enterprise Edition: Oracle's paid, commercially supported edition of MySQL.

Thread groups: independent sub-pools inside the thread pool, each handling a share of the connections (MySQL default 16).

Listener thread: a thread in each group that waits for any of the group's connections to send a statement.

Row locks: locks InnoDB takes on individual rows so two transactions can't change the same row at once.

Stalled: a statement that has run longer than the stall limit and no longer stops others in its group from starting.

TP_CONNECTION_ADMIN: a privilege that lets a connection bypass the transaction limit and run in the Admin thread group.

Sysbench: an open-source benchmarking tool; its OLTP read-write test runs short mixed transactions.

InnoDB buffer pool: InnoDB's in-memory cache of table and index pages.

p95 latency: the time within which 95% of requests finish.

STORY 2

Aurora DSQL learns foreign keys without locks

AMAZON AURORA DSQL · FOREIGN KEYS 26 AUG · PARTIAL INDEXES 15 SEP · AWS DEEP DIVE 1 OCT · ~15 MIN READ

When AWS unveiled Aurora DSQL in December 2024, it promised a Postgres-compatible database that spans regions, never goes down and never makes you think about servers. Then developers tried to migrate, and hit a wall of "not supported". No foreign keys was the one that stung. One Reddit commenter put it bluntly at launch: "You can't truly have 'relational' data without foreign keys."[19] This summer AWS shipped them, along with partial indexes, a new kind of join and a pile of other Postgres features. Foreign keys are the interesting part, because DSQL has a design rule that seems to make them impossible: it never takes locks.

WHAT HAPPENED

Aurora DSQL's release notes for this summer read like a checklist of missing Postgres features being ticked off:[17]

- **3 August:** `ALTER TABLE ... ADD CONSTRAINT ... NOT VALID` and `ALTER TABLE ASYNC ... VALIDATE CONSTRAINT`, for adding constraints to big tables without blocking.
- **13 August:** indexes on expressions, such as `lower(title)`.
- **24–25 August:** wider `SELECT ... FOR UPDATE` support, and a new `SELECT ... FOR KEY SHARE`.
- **26 August: foreign key constraints**, plus compression for large text values.
- **27 August:** extended statistics (`CREATE STATISTICS`) to help the planner with correlated columns.
- **11 September: batched nested-loop joins**.
- **15 September: partial indexes**.

AWS's public announcements followed. The What's New post for foreign keys is dated **27 August**, and lists support for the `NO ACTION`, `RESTRICT`, `CASCADE`, `SET NULL` and `SET DEFAULT` actions in all regions where DSQL runs.[18] InfoQ covered it on **28 September**.[19] On **1 October**, the AWS Database Blog published a long explainer of how the foreign keys work and how to design around them.[20] On **2 October**, AWS announced partial indexes.[21]

Luc van Donkersgoed, a principal engineer at Dutch railway operator Nederlandse Spoorwegen, summed up the reaction InfoQ found: "Lack of FKs was the biggest gap between DSQL and 'normal' Postgres, blocking many migrations."[19]

THE BACKGROUND YOU NEED

What DSQL is

Aurora DSQL *≈ AWS's serverless, distributed, PostgreSQL-compatible database, designed to run active-active across multiple regions* entered public preview on 3 December 2024 with "PostgreSQL v16 compatibility", and became

generally available on 27 May 2025.[17,22] AWS says it's "designed for 99.99% single-Region and 99.999% multi-Region availability", and that reads and writes to any regional endpoint are strongly consistent.[22]

It's "Postgres-compatible" in a specific sense. It uses PostgreSQL's query processing on top, but the storage, replication and transaction machinery underneath are all new. That's why each Postgres feature has to be rebuilt for it, rather than simply switched on.

How a DSQL transaction commits

AWS distinguished engineer Marc Brooker described the write path in a 2024 blog post.[23] In short:

1. **Reads happen at a start time.** When your transaction begins, DSQL picks a timestamp and you read everything as of that moment. This is **MVCC** *≈ multi-version concurrency control: keeping several versions of each row so readers see a consistent snapshot without blocking writers*, the same idea Postgres uses.
2. **Writes are just notes.** An `UPDATE` "simply writes down a planned change locally inside the Query Processor". Nothing goes to storage yet.
3. **At COMMIT, the adjudicator decides.** A separate service called the **adjudicator** *≈ DSQL's commit-time conflict checker: it accepts a transaction only if no other transaction wrote the same keys since it started* checks whether any other transaction wrote the same keys between your start time and your commit time.
4. **If you pass, the journal makes it durable.** The transaction goes to the **journal** *≈ DSQL's internal replicated log of committed transactions; once written there, a commit is durable*, you get your success, and storage nodes apply it in the background.



Simplified from Marc Brooker's 'DSQL Vignette: Transactions and Durability' and the DSQL concurrency control docs. QP = Query Processor.

This is **optimistic concurrency control (OCC)** *≈ a way to run concurrent transactions without locks: everyone proceeds, and conflicts are detected at commit, where the loser is aborted and must retry*. The docs: "Instead of using locks, OCC evaluates conflicts at commit time."[24] The loser gets `ERROR: change conflicts with another transaction (OC000) (SQLSTATE 40001)`, the standard Postgres code for a serialization failure, and your app retries.[24]

Brooker argues this is "the provably minimal amount of coordination" for strong consistency, and that it means a client can never hold a lock it shouldn't, say across a garbage-collection pause.[23] The isolation level is fixed: DSQL provides **snapshot isolation** *≈ each transaction sees a consistent snapshot from its start, and two transactions conflict only if they write the same data*, which matches Postgres's `REPEATABLE READ`.[23,25]

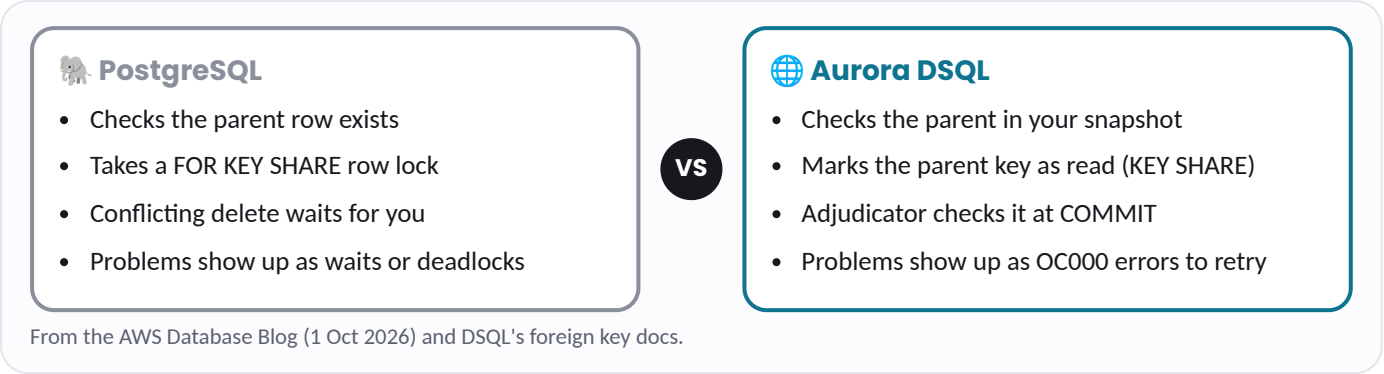
DSQL also limits transactions. One can modify at most 3,000 rows, and Brooker gives the other limits as 10 MiB of writes and five minutes per transaction.[25,26]

Why foreign keys are hard here

A **foreign key** *≈ a constraint that says a column's value must match a key in another table, e.g. every order's product_id must exist in products* check is a *read*. When you insert an order for product 1, the database has to confirm that product 1 exists, and keep it existing until you commit.

Classic Postgres does this with a lock. The AWS blog explains that Postgres takes a **FOR KEY SHARE lock** *≈ a Postgres row lock that says 'I depend on this row's key staying the same'; it blocks deletes and key changes, but not other readers or non-key updates* on the referenced row, so "a transaction that changes key columns or deletes that row waits for transactions still referencing it."[20]

DSQL has no locks to take. And plain snapshot isolation only catches *write-write* conflicts. Brooker noted that serializable isolation would also need to check read-write conflicts, which DSQL doesn't do.[23] So consider two transactions. Session A deletes product 1. Session B inserts an order for product 1. B checked its snapshot, and product 1 was there. A and B wrote *different* rows, so the adjudicator, as originally designed, would let both commit, and you'd have an order pointing to a product that doesn't exist. That's exactly the bug a foreign key exists to prevent.



From the AWS Database Blog (1 Oct 2026) and DSQL's foreign key docs.

HOW IT WORKS

Teaching the adjudicator about reads

The fix came in two steps, and the release notes show the order. On 25 August, DSQL gained `SELECT ... FOR KEY SHARE`, which "declares that a transaction depends on the key columns of the selected rows."[17] It's not a lock. It tells the adjudicator to treat that read as something that can conflict. The next day, foreign keys shipped on top of it.

The documentation describes the two phases.[27]

Phase 1: check against your snapshot. "When you insert or update a referencing row, Aurora DSQL reads the referenced table at your transaction's start-time snapshot to confirm the referenced key exists." Deletes and key updates on the parent check the child table the same way. "Because this verification reads from the transaction start-time snapshot instead of taking a lock, other transactions can continue to modify the referenced and referencing tables in parallel."

Phase 2: settle it at commit. Phase 1 only proves the constraint held when you started. So "Aurora DSQL implicitly applies the `KEY SHARE` clause to referenced rows to detect whether any concurrent change invalidated your snapshot. If Aurora DSQL detects a conflict, it fails the transaction with a serialization error."

Here's the documented example:[27]

```
-- products(1, 'Widget') exists; orders.product_id REFERENCES products
-- Session A                          -- Session B
BEGIN;                                BEGIN;
DELETE FROM products                  INSERT INTO orders
  WHERE product_id = 1;                 VALUES (100, 1, 5);
COMMIT;  -- succeeds
                                      COMMIT;  -- fails:
-- ERROR: change conflicts with another transaction (OC000)
-- (SQLSTATE 40001)
```

The first to commit wins. B's insert passed its snapshot check, but the adjudicator sees that A deleted a row B had marked as a key dependency, and B is rejected. "You can't end up with an order that points to a deleted product."[27]

Brooker summed up the design, quoted by InfoQ: "No blocking, no change in scaling for non-FKC reads."[19] An earlier explainer of the adjudicator by AWS engineer Marc Bowes, written before foreign keys shipped, had sketched the same kind of idea: an "exists" check for inserts that reference a row, which conflicts with deletes but not with other existence checks.[28]

What conflicts, and what doesn't

The clever part is what *doesn't* conflict. A key-share mark only cares about the parent's key. The DSQL concurrency docs give the full matrix:[24]

Your transaction did…	vs. insert / delete / key update / FOR UPDATE	vs. non-key update	vs. FOR KEY SHARE
Insert, delete, key update, FOR UPDATE	conflict	conflict	conflict
Update of non-key columns only	conflict	conflict	no conflict
FOR KEY SHARE (incl. FK checks)	conflict	no conflict	no conflict

The AWS blog spells out what that means for foreign keys:[20]

- Deleting a parent while someone inserts a child that references it: **conflict**.
- Changing a parent's primary key while someone references the old key: **conflict**.
- Changing a column that's in a UNIQUE index on the parent (say, a unique `name`): **conflict**. "Key" here means primary keys and columns of non-partial, non-expression unique indexes.
- Changing a non-key column on the parent, such as `price`: **no conflict**.

- Two sessions inserting children under the same parent: **no conflict**.

That last one matters most. A popular parent row, like a best-selling product, can take many simultaneous child inserts "without false conflicts".[20]

Deferred checks, cascades and existing tables

DSQL supports the full set of Postgres foreign key options: the five referential actions, `MATCH SIMPLE` and `MATCH FULL`, and **deferrable constraints** *≈ foreign keys whose check can be postponed until COMMIT, so a transaction can briefly break the rule, e.g. insert a child before its parent*.[27] With `SET CONSTRAINTS fk_product DEFERRED`, you can insert an order line before its product within one transaction, and the check runs at commit.[20]

Cascades have a DSQL-specific catch. `CASCADE`, `SET NULL` and `SET DEFAULT` change child rows for you, and those changes "count against the 3,000-row transaction mutation limit, so cascading deletes on large child sets fail." AWS recommends `NO ACTION` or `RESTRICT` whenever the number of children is unbounded.[20] (Brooker had already called `ON DELETE CASCADE` a bad idea in a footnote on his blog.[26])

To add a foreign key to a table that already has data, you split it into two steps, so you never scan a big table inside a blocking statement:[20,17]

Adding a foreign key to a big existing table

PostgreSQL habit

```
ALTER TABLE order_items
  ADD CONSTRAINT fk_product
  FOREIGN KEY (product_id)
  REFERENCES products;
-- scans every row now
```

Aurora DSQL

```
ALTER TABLE order_items
  ADD CONSTRAINT fk_product
  FOREIGN KEY (product_id)
  REFERENCES products NOT VALID;
-- new writes checked from now
ALTER TABLE ASYNC order_items
  VALIDATE CONSTRAINT fk_product;
-- runs as a job; see sys.jobs
```

DSQL syntax from the AWS Database Blog (1 Oct 2026) and the DSQL release notes. Each DDL statement must be in its own transaction.

The validation runs as an asynchronous job you can watch in `sys.jobs`.[17] That's important in DSQL, where "a transaction can include only 1 DDL statement" and DDL and DML have to be in separate transactions.[25]

Partial indexes and batched joins

The other two features target performance, and both make sense once you know DSQL's storage is a remote, distributed service rather than a local disk.

A **partial index** *≈ an index over only the rows that match a WHERE condition, so it stays small even as the table grows* is a standard Postgres feature that DSQL added on 15 September: "Add a `WHERE` clause to `CREATE INDEX` to index only the rows that satisfy a predicate." The planner uses it "when it can prove that the query's `WHERE`

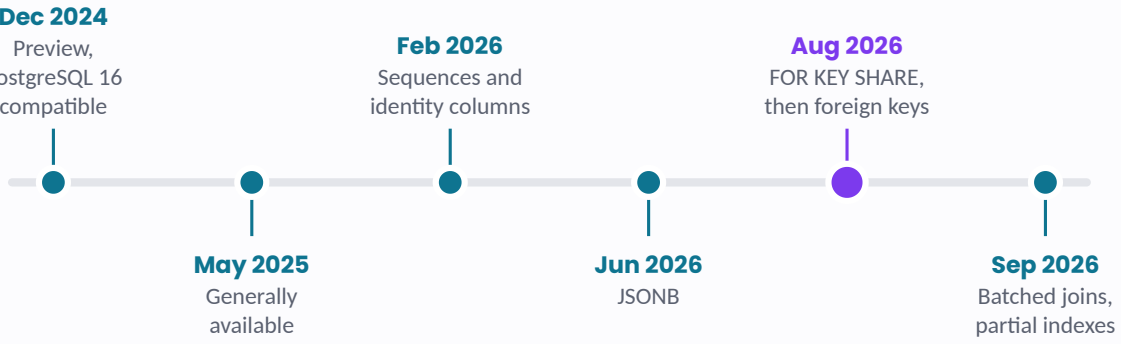
conditions imply the index's predicate."[17] AWS's example is a table with a small working set and a long history, such as open orders among years of completed ones.[21] In DSQL, every index build is asynchronous, so the statement looks like this:[29]

```
CREATE INDEX ASYNC orders_open_idx
  ON orders (customer_id)
  WHERE status = 'open';
```

Functions in the predicate must be immutable, the same rule as for index expressions.[29] AWS's own `dsql-lint` migration checker stopped flagging partial indexes as incompatible in a change merged on 16 September.[30]

A **nested-loop join** *≈ a join that, for each row from one input (the outer side), looks up matching rows in the other (the inner side)* is ideal when the outer side is small and the inner side has an index. But in DSQL, each inner lookup is a round trip from the query processor to storage. A **batched nested-loop join** *≈ a nested-loop join that collects many outer rows first and looks up the inner side for the whole batch at once, cutting round trips* "collects a batch of outer rows, builds the inner scan work for the entire batch, and then joins the returned inner rows back to the matching outer rows." It shows up in `EXPLAIN` as `Nested Loop (Batched Join)`, and you can switch it off per session with `SET dsql.enable_batched_nestloop = off;`.[31]

Aurora DSQL's road to 'normal' Postgres



Dates from the Aurora DSQL release notes and the GA announcement.

WHAT IT MEANS FOR YOU

If foreign keys were your blocker, the migration story just got a lot better. You can bring your schema across with its constraints instead of rewriting referential integrity into application code. AWS's blog suggests two paths for data: load parents before children, or load everything without foreign keys and then add them with `NOT VALID` and an async `VALIDATE`.[20] ORMs are catching up too: the Drizzle ORM adapter added on 25 August includes opt-in OCC retries.[17]

Retries go from "nice to have" to "required". In Postgres, a foreign key conflict means a short wait. In DSQL it means an error. AWS's guidance:[20]

- `40001` / `OC000` (data conflict): retry the whole transaction from `BEGIN`, with exponential backoff and jitter and a cap on attempts.

- `OC001` (the session's cached schema is out of date): usually succeeds on the first retry.
- `23503` (an actual foreign key violation): don't retry. That's a bug in your data or logic.

Design around hot parents. Ordinary child inserts don't conflict with each other, but anything that touches a parent's *key* does. AWS's advice is to keep referenced keys stable and move frequently changing values into non-key columns.[27] Its blog also warns about "scale-explosive" relationships, such as every audit log row pointing to one tenant, or every engagement row to one viral post. There, avoid `CASCADE`, soft-delete the parent and clean up children in batches of 3,000 or fewer, and copy mutable attributes like price into the child row when you write it.[20] If you see a steady rise in 40001 errors, AWS says it's a sign of a hot parent row.[20]

Benchmark first. "All data manipulation language (DML) operations on referenced or referencing tables incur extra reads to guarantee referential integrity." AWS's advice is to benchmark before adding a constraint.[27] It hasn't published numbers for that overhead.

> 💥 **Watch out for**
>
> - **No performance numbers yet.** Neither the docs nor the 1 October blog post quantify the cost of the extra reads or the conflict rate under load.[20]
> - **Updating a unique column on a parent conflicts with child inserts.** That includes columns that seem harmless, such as a unique `name` or `email`, not just the primary key.[20]
> - **Cascades hit the 3,000-row limit.** A `DELETE` that cascades to 5,000 children fails.[20]
> - `SET CONSTRAINTS` **only affects foreign keys.** Primary key, unique, `NOT NULL` and `CHECK` constraints are always checked immediately.[20]
> - **It's still not identical to Postgres.** Isolation is fixed at Repeatable Read, `FOR SHARE` and `FOR NO KEY UPDATE` aren't supported, and indexes must be built with `CREATE INDEX ASYNC`.[25,24,29] Two concurrent updates to the same row conflict even if they change different non-key columns.[24]

WHAT HAPPENS NEXT

The pace suggests more is coming. Since June, DSQL's release notes have added a Postgres feature roughly every week or two.[17] Meanwhile, if you migrate, the AWS blog post is the best guide so far to which relationships will scale and which will fight you.[20]

📖 Words from this story

Aurora DSQL: AWS's serverless, distributed, PostgreSQL-compatible database, designed to run active-active across regions.

MVCC: multi-version concurrency control; keeping several versions of each row so readers see a consistent snapshot without blocking writers.

Adjudicator: DSQL's commit-time conflict checker.

Journal: DSQL's internal replicated log of committed transactions.

Optimistic concurrency control (OCC): running transactions without locks and detecting conflicts at commit, where the loser must retry.

Snapshot isolation: each transaction sees a consistent snapshot from its start; transactions conflict only if they write the same data.

Foreign key: a constraint that a column's value must match a key in another table.

FOR KEY SHARE lock: a Postgres row lock meaning "I depend on this row's key staying the same".

Deferrable constraints: foreign keys whose check can wait until COMMIT.

Partial index: an index over only the rows that match a WHERE condition.

Nested-loop join: a join that looks up matching inner rows for each outer row.

Batched nested-loop join: a nested-loop join that looks up the inner side for many outer rows at once, cutting round trips.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19 RC1 is scheduled for today:** the open-items wiki lists RC1 on 15 October, RC2 on 12 November and GA planned for 19 November, with three open items left.[32]
- **Kafka 4.4.0 is still not out:** the newest post on the Apache Kafka blog is still the 4.2.2 bug-fix release of 29 September.[33]
- **DuckDB 2.0 is a week away:** the release calendar still lists 2.0.0 on 21 October and 2.0.1 on 18 November, both tentative.[34]
- **BigQuery's Postgres connector goes incremental:** incremental transfers for the PostgreSQL and ServiceNow connectors became GA on 12 October, and the Lakehouse runtime catalog's Apache Hive endpoint became GA on 14 October.[35]
- **Confluent Cloud (9–14 Oct):** its unified stream manager can now show KRaft controller health for Confluent Platform clusters, and two new Debezium transforms help CDC connectors flag changed fields and route many tables to one topic.[36]
- **Snowflake (12–13 Oct):** Horizon Context and a Cortex Agent SDK entered preview.[37]
- **ClickHouse goes hunting a 32 ms tail:** ClickHouse's engineering blog published "Chasing that 32 ms tail latency in ClickHouse" on 13 October.[38]

1. **MySQL July 2026 GA Releases Now Available**, Mike Frank, Oracle MySQL blog, 31 Jul 2026.
2. **Changes in MySQL 26.7.0 (2026-07-28)**, MySQL 26.7 Release Notes, Oracle.
3. **Changes in MySQL 9.7.2 (2026-07-28)**, MySQL 9.7 Release Notes, Oracle.
4. **Thread Pool in Percona Server and MySQL (Part 1)**, Bogdan Degtyariov, Percona blog, 5 Oct 2026.
5. **Thread Pool in Percona Server and MySQL (Part 2)**, Bogdan Degtyariov, Percona blog, 12 Oct 2026.
6. **Meet MySQL 26.7: A New Versioning Era Begins**, Anil Joshi, Percona blog, 12 Oct 2026.

7. **MySQL Thread Pool**, MySQL 26.7 Reference Manual, section 7.6.3, viewed 15 Oct 2026.
8. **MySQL 9.7 FAQ: MySQL Enterprise Thread Pool**, MySQL 9.7 Reference Manual, appendix A.15, viewed 15 Oct 2026.
9. **MySQL Enterprise Thread Pool Overview**, MySQL 9.7 Reference Manual, viewed 15 Oct 2026.
10. **Thread Pool in MariaDB**, MariaDB Knowledge Base.
11. **Thread pool**, Percona Server for MySQL 8.4 documentation, viewed 15 Oct 2026.
12. **MySQL 9.7 Is Out and the Community Wins**, Lenka Kasparova and Alkin Tezuysal, Oracle MySQL blog, 6 May 2026.
13. **Thread Pool Operation**, MySQL 9.7 Reference Manual, section 7.6.3.3, viewed 15 Oct 2026.
14. **Thread Pool Tuning**, MySQL 26.7 Reference Manual, section 7.6.3.4, viewed 15 Oct 2026.
15. **The New MySQL Thread Pool**, Daniel Blanchard, MySQL Blog Archive, 7 Jun 2023.
16. **Thread Pool Installation**, MySQL 8.4 Reference Manual, section 7.6.3.2, viewed 15 Oct 2026.
17. **Release notes for Aurora DSQL**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
18. **Amazon Aurora DSQL now supports foreign key constraints**, AWS What's New, 27 Aug 2026.
19. **AWS Introduces Foreign Key Constraints in Aurora DSQL**, Renato Losio, InfoQ, 28 Sep 2026.
20. **Working with foreign key constraints in Aurora DSQL**, Rekha Reddy Anupati and Arnab Chowdhury, AWS Database Blog, 1 Oct 2026.
21. **Amazon Aurora DSQL now supports partial indexes**, AWS What's New, 2 Oct 2026.
22. **Amazon Aurora DSQL is now generally available**, AWS What's New, 27 May 2025.
23. **DSQL Vignette: Transactions and Durability**, Marc Brooker, Marc's Blog, Dec 2024.
24. **Concurrency control in Aurora DSQL**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
25. **Migrating from PostgreSQL to Aurora DSQL**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
26. **DSQL: Simplifying Architectures**, Marc Brooker, Marc's Blog.
27. **Working with foreign key constraints in Aurora DSQL**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
28. **Aurora DSQL: The Adjudicator**, Marc Bowes (undated).
29. **CREATE INDEX**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
30. **fix(dsql-lint): accept partial indexes (#172)**, awslabs/aurora-dsql-tools, merged 16 Sep 2026.
31. **Batched nested-loop joins in Aurora DSQL EXPLAIN plans**, Amazon Aurora DSQL User Guide, viewed 15 Oct 2026.
32. **PostgreSQL 19 Open Items**, PostgreSQL wiki, viewed 15 Oct 2026.
33. **Apache Kafka blog**, viewed 15 Oct 2026.
34. **DuckDB release calendar**, viewed 15 Oct 2026.
35. **BigQuery release notes**, Google Cloud, entries dated 12 and 14 Oct 2026.
36. **Release Notes for Confluent Cloud**, Confluent, entries dated 9–14 Oct 2026.
37. **All release notes**, Snowflake documentation, entries dated 12–13 Oct 2026.
38. **ClickHouse blog**, post listing viewed 15 Oct 2026.